

A palindrome is a string that reads the same from left to right as it does from right to left.

Given a string S of N lowercase English letters, we define a k -length rotation as cutting the first k characters from the beginning of S and appending them to the end of S . For each S , there are N possible k -length rotations (where $0 \leq k < N$). See the Explanation section for examples.

Given N and S , find all N k -length rotations of S ; for each rotated string, S_k , print the maximum possible length of any palindromic substring of S_k on a new line.

Input Format

The first line contains an integer, N (the length of S).
The second line contains a single string, S .

Constraints

- $1 \leq N \leq 5 \times 10^5$
- $0 \leq k < N$
- S is comprised of lowercase English letters.

Output Format

There should be N lines of output, where each line k contains an integer denoting the maximum length of any palindromic substring of rotation S_k .

Sample Input 0

```
13
aaaaabbbbbaaaa
```

Sample Output 0

```
12
12
18
8
8
9
11
13
13
```

☐ Test against custom input



You have earned 120.00 points!

You are now 360 points away from the gold level for your problem solving badge.

4%

490/650

Congratulations

You solved this challenge. Would you like to challenge your friends?



☒ Test case 0

☒ Test case 1

☒ Test case 2

☒ Test case 3

☒ Test case 4

☒ Test case 5

☒ Test case 6

Compiler Message:

Success

Input (stdin)

```
1 13
2 aaaaabbbbbaaaa
```

Expected Output:

```
1 12
2 12
3 18
4 8
```

Given a string of lowercase letters in the range `ascii[a-z]`, determine the index of a character that can be removed to make the string a *palindrome*. There may be more than one solution, but any will do. If the word is already a palindrome or there is no solution, return `-1`. Otherwise, return the index of a character to remove.

Example

`s = "bcbcb"`

Either remove 'b' at index 0 or 'c' at index 3.

Function Description

Complete the `palindromeIndex` function in the editor below.

`palindromeIndex` has the following parameter(s):

- string `s`: a string to analyze

Returns

- int: the index of the character to remove or `-1`

Input Format

The first line contains an integer `q`, the number of queries.

Each of the next `q` lines contains a query string `s`.

Constraints

- $1 \leq q \leq 20$
- $1 \leq \text{length of } s \leq 10^5 + 5$
- All characters are in the range `ascii[a-z]`.

Sample Input

STDIN	Function
3	q = 3
aaab	s = 'aaab' (first query)
baa	s = 'baa' (second query)
aaa	s = 'aaa' (third query)

Sample Output

```

49     System.out.println(palindromeIndex(s));
50     }
51
52     bufferedReader.close();
53     }
54
55 }
    
```

Line: 55 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Compiler Message

Success

Input (stdin)

```

1 3
2 aaab
3 baa
4 aaa
    
```

Download

Your Output (stdout)

```

1 3
2 0
    
```

A space explorer's ship crashed on Mars! They send a series of SOS messages to Earth for help.



Letters in some of the SOS messages are altered by cosmic radiation during transmission. Given the signal received by Earth as a string, s , determine how many letters of the SOS message have been changed by radiation.

Example

$s = \text{"SOSTOT"}$

The original message was SOS SOS. Two of the message's characters were changed in transit.

Function Description

Complete the marsExploration function in the editor below.

marsExploration has the following parameter(s):

- string s : the string as received on Earth

Returns

- int: the number of letters changed during transmission

Input Format

There is one line of input: a single string, s .

Constraints

- $1 \leq \text{length of } s \leq 99$
- $\text{length of } s \bmod 3 = 0$
- s will contain only uppercase English letters, ascii[A-Z].

Sample Input 0

```
SOSPSSQSSOR
```

Sample Output 0

```
3
```

```
30 }
31 }
32 }
```

Line: 32 col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

Download

Sample Test case 1

```
1 SOSSPSSQSSOR
```

Sample Test case 2

Your Output (stdout)

```
1 3
```

Expected Output:

```
1 3
```

Download

Problem

Submissions

Leaderboard

Discussions

Editorial

Given two strings, determine if they share a common substring. A substring may be as small as one character.

Example

$s1 = \text{'aaid'}$

$s2 = \text{'art'}$

These share the common substring a .

$s1 = \text{'be'}$

$s2 = \text{'cab'}$

These do not share a substring.

Function Description

Complete the function `twoStrings` in the editor below.

`twoStrings` has the following parameter(s):

- string $s1$: a string
- string $s2$: another string

Returns

- string: either YES or NO

Input Format

The first line contains a single integer p , the number of test cases.

The following p pairs of lines are as follows:

- The first line contains string $s1$.
- The second line contains string $s2$.

Constraints

- $s1$ and $s2$ consist of characters in the range `ascii[a-z]`.
- $1 \leq p \leq 10$
- $1 \leq |s1|, |s2| \leq 10^5$

Output Format

For each pair of strings, return YES or NO.

Sample Input

```
57
58     bufferedReader.close();
59     bufferedWriter.close();
60 }
61
62
```

Lines: 62 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

Download

Sample Test case 1

```
1 2
2 hello
3 world
4 hi
5 world
```

Sample Test case 2

Your Output (stdout)

```
1 YES
2 NO
```

Expected Output

Download

For two strings A and B, we define the similarity of the strings to be the length of the longest prefix common to both strings. For example, the similarity of strings "abc" and "abd" is 2, while the similarity of strings "aaa" and "aaab" is 3. Calculate the sum of similarities of a string S with each of its suffixes.

Input Format

The first line contains the number of test cases t .
Each of the next t lines contains a string to process, s .

Constraints

- $1 \leq t \leq 10$
- $1 \leq |s| \leq 100000$
- s is composed of characters in the range `ascii[a-z]`

Output Format

Output t lines, each containing the answer for the corresponding test case.

Sample Input

```
2
ababaa
aa
```

Sample Output

```
11
3
```

Explanation

For the first case, the suffixes of the string are "ababaa", "babaa", "abaa", "baa", "aa" and "a". The similarities of these strings with the string "ababaa" are 6,0,3,0,1, & 1 respectively. Thus, the answer is $6 + 0 + 3 + 0 + 1 + 1 = 11$.

For the second case, the answer is $2 + 1 = 3$.

```
54         sum += z[i];
55     }
56
57     return sum;
58 }
59
```

Upload Code as File

☐ Test against custom input

Run C

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

Input (stdin)

```
1 2
2 ababaa
3 aa
```

Your Output (stdout)

```
1 11
2 3
```

Expected Output

```
1 11
2 3
```

Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots

Markup and

Accepted 144 / 144 testcases passed
kimidibinhu2 submitted at Aug 29, 2020 14:57

Analysis

Solution

7583.25 / 7584
Java user 88%

Unlock the Full LeetCode Experience

Company problems, Ask Leet and expert editorials — all in one plan.

Runtime

14 ms | Beats 90.57%

Memory

43.35 MB | Beats 89.52%



Code Java

```
1 class Solution {
2     public String longestPalindrome(String s) {
3         if (s == null || s.length() < 1) {
4             return "";
5         }
6
7         int start = 0;
8         int end = 0;
```

View more

More challenges

Java Auto

```
17
18 // If we found a longer palindrome, update start and end indices
19 if (len > end - start) {
20     start = i - (len - 1) / 2;
21     end = i + (len - 1) / 2;
22 }
23
24 return s.substring(start, end + 1);
25
26
27 // Helper method to expand outwards from the center and return palindrome length
28 private int expandAroundCenter(String s, int left, int right) {
29     while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
30         left--;
31         right++;
32     }
33     // Length of the valid palindrome found
34     return right - left - 1;
35 }
36
37 }
```

Saved

In 37, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

s =
"babab"

Output

"aba"

Expected

"aba"

Description Accepted Editorial Solutions Submissions

All Submissions

Accepted 56 / 66 testcases passed. Time taken: 29d 22hrs 3m 32s
kimidibindhu2 submitted at Aug 29, 2026 14:50

Analysis Solution

7583.25 ms
Beat over 65%

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime Memory
0 ms Beats 100.00% 43.26 MB Beats 18.42%

Code Java

```
1 class Solution {
2     public int strStr(String haystack, String needle) {
3         int hLen = haystack.length();
4         int nLen = needle.length();
5
6         // If needle is longer than haystack, it cannot be found
7         if (hLen < nLen) {
8             return -1;
9         }
10
11        // Loop through haystack up to the point where needle can still fit
12        for (int i = 0; i <= hLen - nLen; i++) {
13            // Check if the substring matches needle
14            if (haystack.substring(i, i + nLen).equals(needle)) {
15                return i;
16            }
17        }
18
19        return -1;
20    }
21 }
```

View more

More challenges

</> Code

Java Auto

```
1 class Solution {
2     public int strStr(String haystack, String needle) {
3         int hLen = haystack.length();
4         int nLen = needle.length();
5
6         // If needle is longer than haystack, it cannot be found
7         if (hLen < nLen) {
8             return -1;
9         }
10
11        // Loop through haystack up to the point where needle can still fit
12        for (int i = 0; i <= hLen - nLen; i++) {
13            // Check if the substring matches needle
14            if (haystack.substring(i, i + nLen).equals(needle)) {
15                return i;
16            }
17        }
18
19        return -1;
20    }
21 }
```

Saved In 21

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

haystack =
"sadbutsad"

needle =
"sad"

Output

0

Accepted 65 / 65 testcases passed

kindibind... submitted at Aug 28, 2026 14:44

Analysis

Sync w/ LeetCode

Solution

4583.25ms
Runtime 99%

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

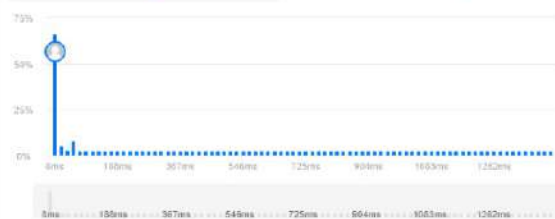


Runtime

9 ms | Beats 92.28%

Memory

46.78 MB | Beats 71.57%



Code / Java

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 class Solution {
5     public List<Integer> findAnagrams(String s, String p) {
6         List<Integer> result = new ArrayList<>();
7
8         if (s.length() < p.length()) {
```

[View more](#)[More challenges](#)

Java Auto

```
38     }
39     }
40
41     return result;
42 }
43
44 // Helper method to compare two frequency arrays
45 private boolean matches(int[] a, int[] b) {
46     for (int i = 0; i < 26; i++) {
47         if (a[i] != b[i]) {
48             return false;
49         }
50     }
51     return true;
52 }
53 }
```

Saved

Testcase Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

```
s =
"cbaebabacd"
```

```
p =
"abc"
```

Output

```
[0,6]
```

Expected

```
[0,6]
```

Problem List

Description Accepted Editorial Solutions Submissions

All Submissions

Accepted 511 / 511 testcases passed

kimidbindu2 submitted at Aug 29, 2025 14:40

Analysis Solution

7583.25 ms (Beats 88%)

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime: 2 ms | Beats 65.03%

Memory: 43.14 MB | Beats 92.36%

Code / Java

```
1 class Solution {
2     public boolean rotateString(String s, String goal) {
3         // If lengths don't match, goal cannot be formed by shifting s
4         if (s.length() != goal.length()) {
5             return false;
6         }
7
8         // Concatenate s with itself and check if goal is a substring
9         return (s + s).contains(goal);
10    }
11 }
```

View more

More challenges

Code

Java Auto

```
1 class Solution {
2     public boolean rotateString(String s, String goal) {
3         // If lengths don't match, goal cannot be formed by shifting s
4         if (s.length() != goal.length()) {
5             return false;
6         }
7
8         // Concatenate s with itself and check if goal is a substring
9         return (s + s).contains(goal);
10    }
11 }
```

Saved

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

s = "ab"

goal = "ba"

Output

false

Expected

false

Problem List

Description

Accepted

Editorial

Solutions

Submissions

Accepted 133 / 133 testcases passed
kimidbnd... submitted at Aug 29, 2020 14:35

Analysis

Sync w/ LeetCode

Solution

7583.25 ms

Unlock the Full LeetCode Experience

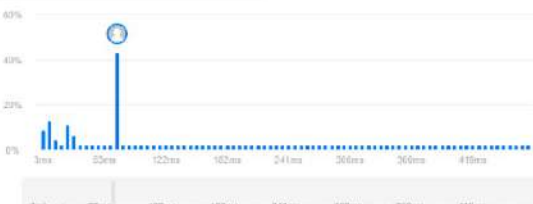
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

75 ms | Beats: 45.56%

Memory

46.51 MB | Beats: 82.36%



Code: Java

```
1 class Solution {
2     public boolean repeatedSubstringPattern(String s) {
3         String doubled = s + s;
4         String sub = doubled.substring(1, doubled.length() - 1);
5         return sub.contains(s);
6     }
7 }
8
```

More challenges

Submit

Code

Java

Auto

```
1 class Solution {
2     public boolean repeatedSubstringPattern(String s) {
3         String doubled = s + s;
4         String sub = doubled.substring(1, doubled.length() - 1);
5         return sub.contains(s);
6     }
7 }
8
```

Testcase

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s = "abab"

Output

true

Expected

true

Contribute a testcase